

GAN Dashboard

DATS 6303: Final Project

Jeongwon Yoo, Pratik Shetty, Josh Schechter

Generative AI models have been increasingly improving over the years. For our final project, we sought a way to explore some of the backbone generative AI models, specifically for image generation. We selected to explore different variations of the Generative Adversarial Network (GAN) Architecture. We started with a simple DCGAN architecture, then worked on a WGAN-GP implementation, and finally attempted to train a ProGAN model. Once training was complete, we put together an interactive dashboard using streamlit, to display our results and demonstrate our understanding of the architectures. Our evaluation metric was Frechet Inception Distance (FID), which we selected because it both tracks diversity of its own images, and also the distance from a real image.

My section of the project was focused on the implementation and development of the WGAN-GP model architecture and training. We each took one model and focused on that model. I wrote the classes for the WGAN-GP model, and I also did extensive background research on DCGAN and Vanilla GAN to allow for me to have a proper understanding of how GAN's really work, and how they can be enhanced. I also read the paper on ProGAN, as I wanted to have a good understanding of the other model being developed, although I did not have a large hand in any of the training done on that.

I also took responsibility for getting the data pipeline set up, and also getting the framework for the UI set up. I wanted to ensure that we were all working on the correct sections, so I tried to set up the repository and pipelines in a way that would allow for easy collaboration.

Architecture

The Wasserstein GAN with Gradient Penalty consists of a generator and a critic, where the critic replaces the discriminator and outputs a real-valued score instead of a probability.

The generator maps a latent noise vector to an image using a series of transposed convolution layers. Each layer upsamples the feature maps, with BatchNorm and ReLU activations, until reaching the desired image resolution. A final Tanh activation produces the output image. The critic is a convolutional network that downsamples the input image through strided convolutions and LeakyReLU activations. It outputs a single scalar score, representing how “real” the image is, without using a sigmoid function.

A key component is the gradient penalty, which enforces the Lipschitz constraint:

$$\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}, \quad \epsilon \sim U(0, 1)$$

$$\mathcal{L}_{GP} = \lambda \mathbb{E}_{\hat{x}} (\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2$$

$$\mathcal{L}_C = \mathbb{E}_{\tilde{x} \sim P_g} [D(\tilde{x})] - \mathbb{E}_{x \sim P_r} [D(x)] + \mathcal{L}_{GP}$$

This stabilizes training by penalizing deviations of the gradient norm from 1 on interpolated samples between real and generated images. Overall, WGAN-GP improves training stability and sample quality compared to standard GANs.

Training

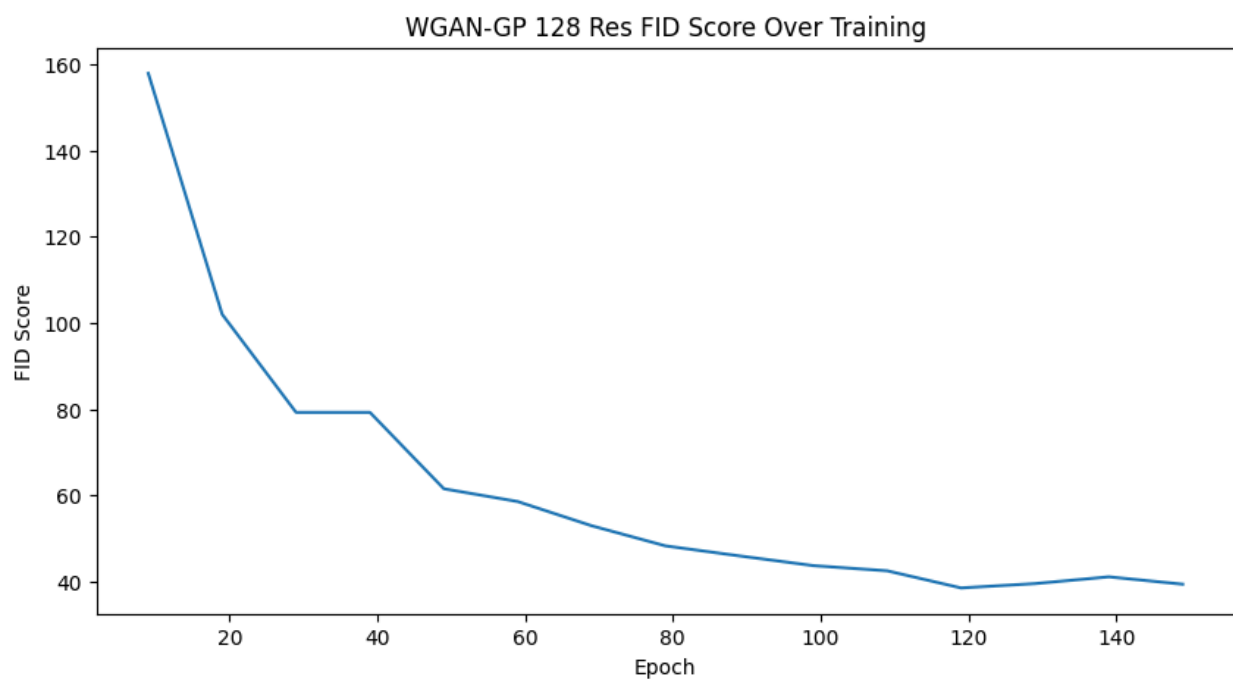
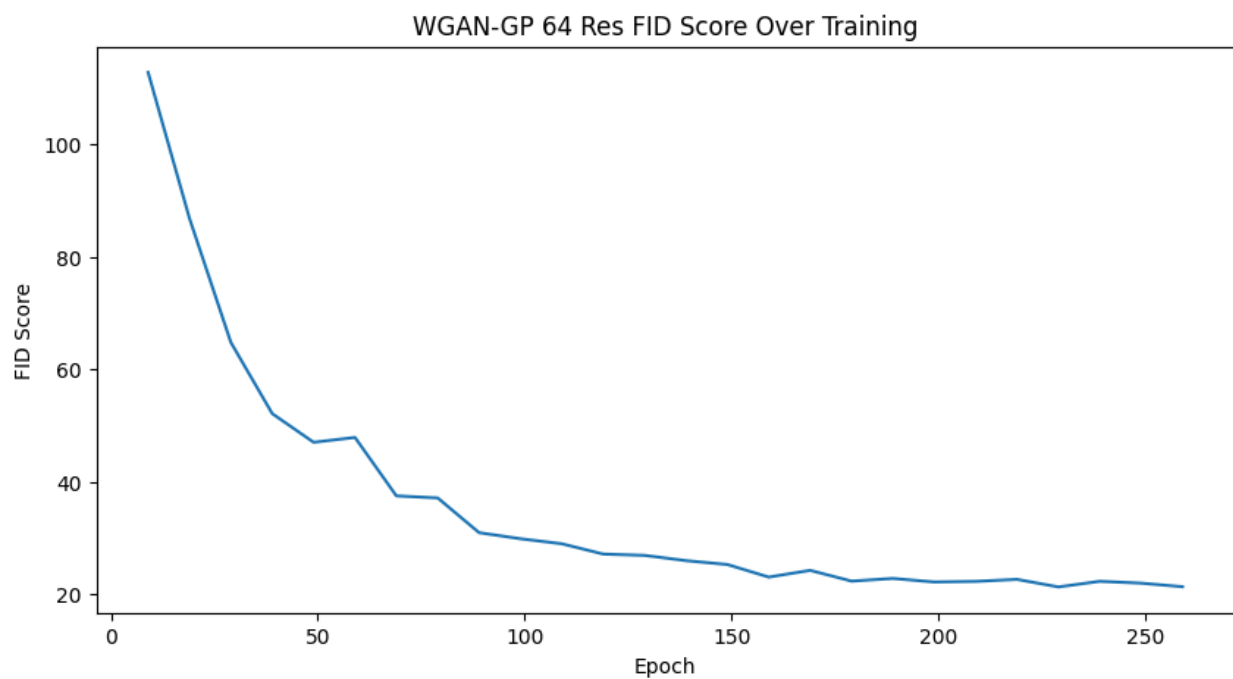
The WGAN-GP model was trained using separate Adam optimizers for the generator and critic. During each batch, the critic was updated first by comparing its scores on real images and generated images. The critic loss was defined as the Wasserstein loss plus the gradient penalty term, which helps enforce the Lipschitz constraint and stabilize training.

The generator was updated every n_{critic} updates. Its objective was to generate images that received higher critic scores, using the loss:

$$\mathcal{L}_G = -\mathbb{E}_{z \sim p(z)} [D(G(z))]$$

During generator updates, the critic parameters were temporarily frozen to avoid unnecessary gradient computation. For full training runs, I used cosine annealing to gradually reduce the learning rate and evaluated the model with FID every 10 epochs. The best checkpoint was saved based on the lowest FID score, with early stopping applied if FID did not improve for three evaluation periods.

The figure below shows the healthy training dynamics for the 64 and 128 image resolution WGAN-GP runs.



Conclusions

Upon comparison to other models, in our experiment it turned out that WGAN-GP ended up having the best all around results. This was expected over DCGAN, but it was surprising that it performed better than ProGAN, even at higher resolutions. A few theories could be had here. First, the image resolutions may not have been large enough for ProGAN to become beneficial enough. Second, ProGAN is extremely difficult to train and tune, especially over a short period of time. If given more time, it is very possible that ProGAN would have outperformed at an image resolution of 128.

Code Percentages

None of my code was taken directly from any source on the internet. However, when stuck modifications were made based on existing coded solutions. One code snippet that comes to mind is the following:

```
if (i + 1) % params[
    'n_critic'] == 0: # this block is only entered every n_critic steps ->
    this replaced the third loop
```

This was in the WGAN-GP train loop. It was to ensure that the number of critic updates outweighed the number of generator updates properly. Instead of nesting for loops that would have been overcomplicated, there was an existing solution I found that computed the logic this way.

Additionally, many helper functions were developed with the assistance of generative AI. None of my scripts or ideas were taken from AI. However, whenever I would need a small task, that could take up more time than is really necessary, I would sometimes use AI to get me started on the logic, then fit it to ensure that it works with my codebase. Additionally, our group used copilot to review our merge requests on Github. We did not solely rely on copilot, as I believe that peer reviews are important so

everybody can learn even from tasks that they are not assigned to. Overall, here is the code percentage:

utils: 75% used AI assistance

[train.py](#): ~5% used AI assistance

Wgan_gp architecture: 18.75% used either AI assistance or referenced an existing github repo (Emillian Dupont)

Overall from [mywork.py](#):

Code_assistance = 279 / 1663 = 0.168

So about 16.8% of my code was developed with the assistance of AI, or the github repo in references.

References

- Goodfellow et al. (2014), *Generative Adversarial Nets*
<https://arxiv.org/pdf/1406.2661>
- Radford et al. (2015), *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*
<https://arxiv.org/pdf/1511.06434>
- Arjovsky et al. (2017), *Wasserstein GAN*
<https://arxiv.org/abs/1701.07875>
- Gulrajani et al. (2017), *Improved Training of Wasserstein GANs*
<https://arxiv.org/abs/1704.00028>

- Karras et al. (2017), *Progressive Growing of GANs for Improved Quality, Stability, and Variation*
<https://arxiv.org/abs/1710.10196>
- Zeiler and Fergus (2010), *Deconvolutional Networks*
<https://www.matthewzeiler.com/mattzeiler/deconvolutionalnetworks.pdf>
- EmilienDupont, *wgan-gp* GitHub repository
Consulted for reference while implementing WGAN-GP ideas; code was not copied or directly reused.
<https://github.com/EmilienDupont/wgan-gp>